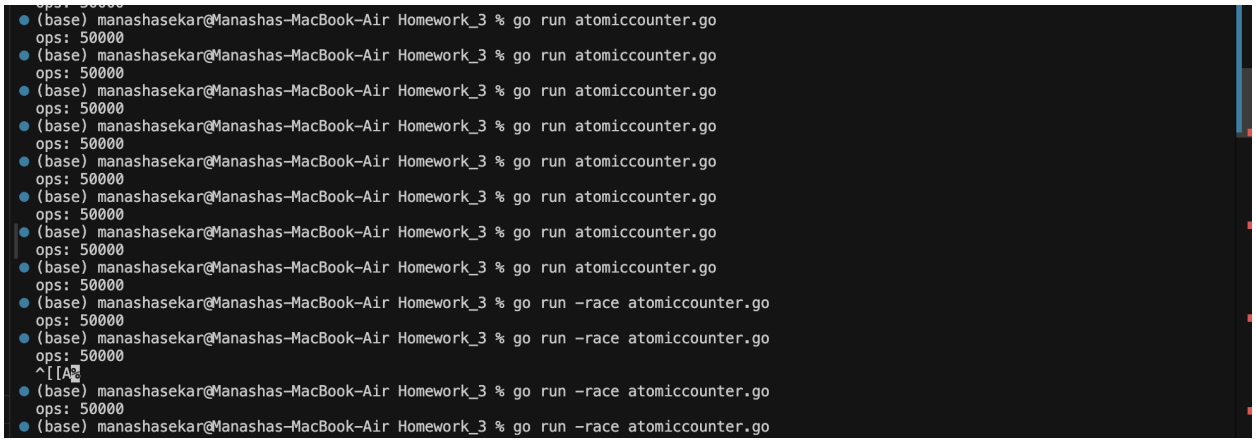
PART II: Thread Experiments in Go

Atomicity:

- **With Atomic integer**, we get a stable output and there are no race conditions.
- Atomic operations ensure only one goroutine modifies the counter at a time.



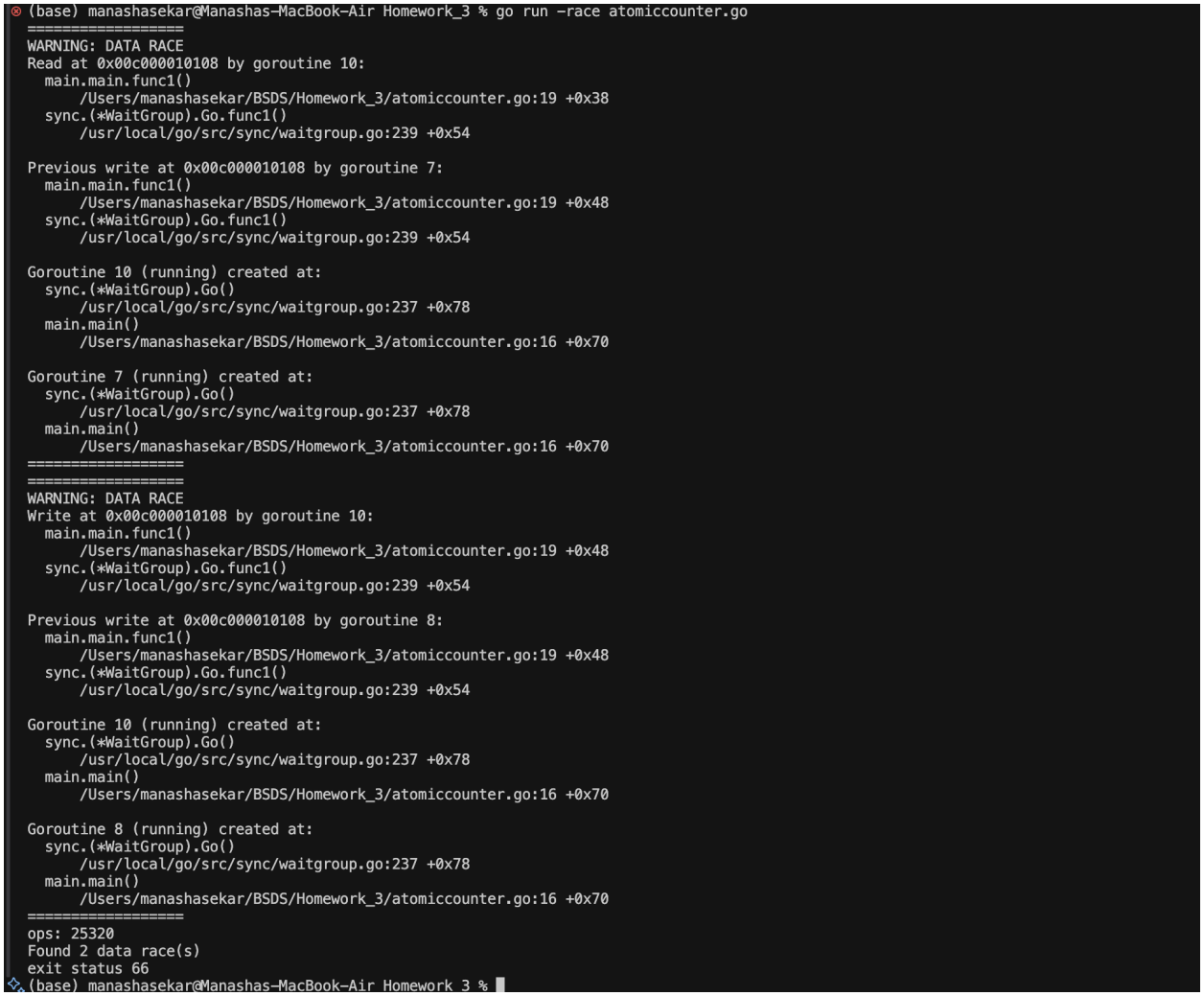
Without Atomic Integer

The Go routines run simultaneously and fails to update.
When one Go routine reads the other updates so in the process it fails to update and we have a few race conditions (multiple goroutines access the same data at the same time). There is **no synchronization** between them and the result is **unpredictable.**

The -race flag revealed:

- Goroutine 8 READING the value while Goroutine 7 was WRITING
- Goroutine 10 WRITING while Goroutine 8 was WRITING
- Multiple concurrent accesses to the same memory location (0x00c000010108)

This shows the reads and writes are happening unsynchronized.



Collections:

Maps in Go are not thread safe, So While all the maps try to create key-value in the same memory the program crashes.

The program crashed with error: `fatal error: concurrent map writes`

- When 50 goroutines simultaneously tried to write key-value pairs to the same map in memory, the map's internal data structure became corrupted.
- Even though each goroutine writes to different keys, they all modify the same map's internal structure (hash table).

```
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_unsafe.go
fatal error: concurrent map writes

goroutine 26 [running]:
internal/runtime/maps.fatal({0x10417ecbb?, 0x0?})
        /usr/local/go/src/runtime/panic.go:1046 +0x20
main.main.func1(0x7)
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:19 +0x48
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 1 [runnable]:
main.main()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x8c

goroutine 19 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 20 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 21 [runnable]:
internal/runtime/maps.typedmemmove(0x1041b2640?, 0x1400010a1a8?, 0x140001081b8?)
        /usr/local/go/src/runtime/mbarrier.go:235 +0x38
internal/runtime/maps.(*table).uncheckedPutSlot(0x14000102040, 0x1041b8aa0?, 0x95b8746314142de5, 0x1041c6e00?, 0x140001081b8?)
        /usr/local/go/src/internal/runtime/maps/table.go:409 +0x140
internal/runtime/maps.(*table).grow(0x14000102020, 0x1041b8aa0, 0x140000a20f0, 0x80?)
        /usr/local/go/src/internal/runtime/maps/table.go:1224 +0x190
internal/runtime/maps.(*table).rehash(0x140000a20f0?, 0x1041b8aa0?, 0x0?)
        /usr/local/go/src/internal/runtime/maps/table.go:1136 +0x30
main.main.func1(0x2)
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:19 +0x48
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 22 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 23 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 24 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 25 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 27 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

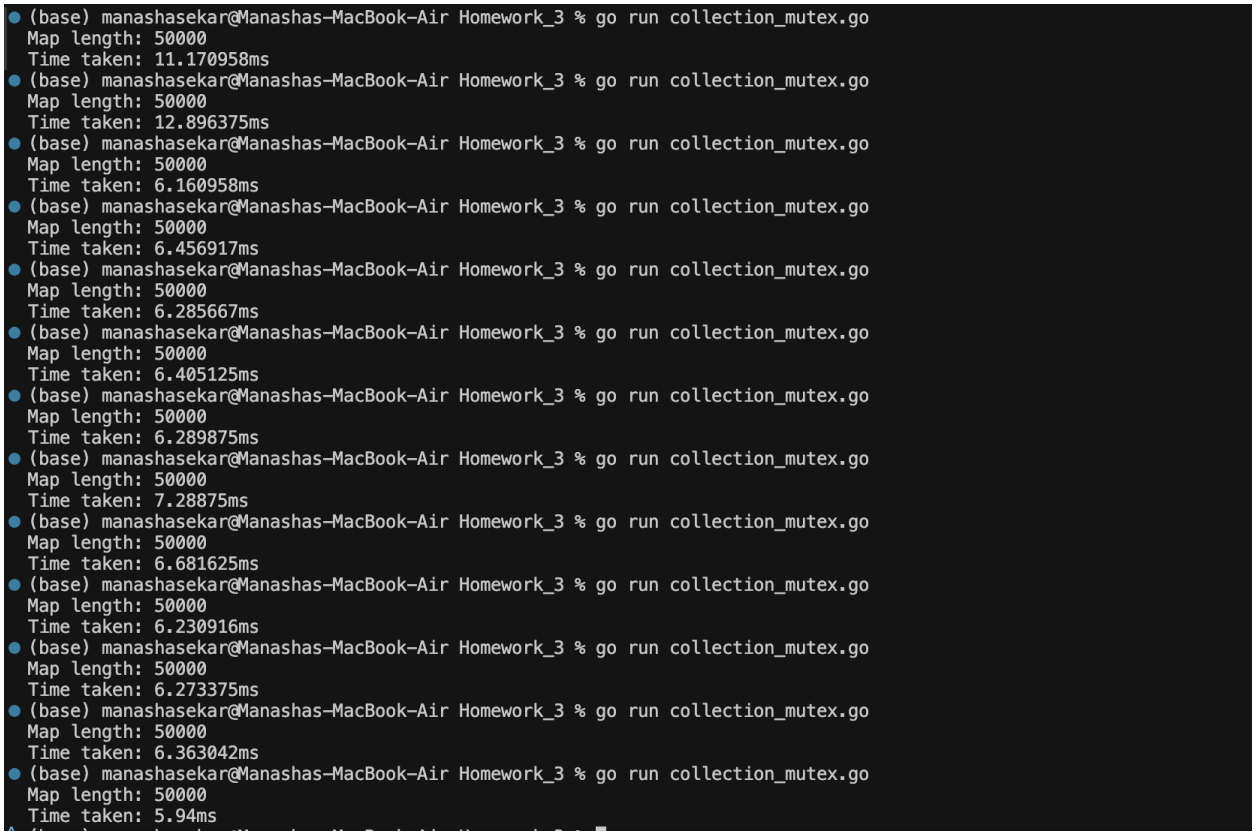
goroutine 28 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16 +0x60

goroutine 29 [runnable]:
main.main.gowrap1()
        /Users/manashasekar/BSDS/Homework_3/collection_unsafe.go:16
runtime.goexit({})
        /usr/local/go/src/runtime/asm_arm64.s:1268 +0x4
```

Mutex

Creates a protected container.

- Each write is protected by Lock/Unlock
- Only one goroutine accesses the map at a time
- No corruption possible



What lesson learned here :

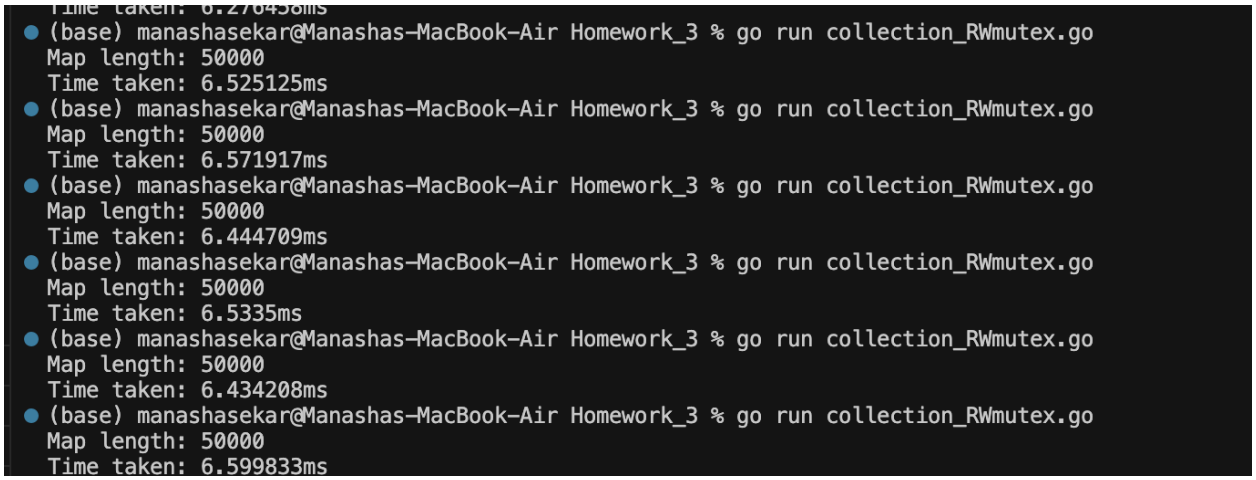
- Without Mutex the Goroutines crash because the concurrent access to map structure corrupts it.
- With the Goroutines the access to limited to one Goroutine at a time.

Mean Time Calculation

Mean = (6.160958 + 6.456917 + 6.285667) / 3 = 18.903542 / 3 = **6.30ms.**

RWMutex

RWMutex took approximately the **same time** as Mutex. In this experiment, all 50 goroutines only WRITE (no reads). Since writes use Lock() (not RLock()), RWMutex behaves identically to regular Mutex. The RLock() optimization is never used.



Mean = (6.444709 + 6.5335 + 6.434208) / 3
= 19.412417 / 3
= 6.47ms (same as Mutex)

- RWMutex is optimized for read-heavy workloads.
- In write-only scenarios, it provides no benefit.

Sync.Map

sync.Map is the fastest because concurrent writes are allowed. Multiple goroutines can write simultaneously in less waiting time. It's Store() has 1 operation (locking built-in).

```
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.692084ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.56075ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.605167ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.338625ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.646542ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.709125ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.936834ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.589458ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.49025ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.44375ms
(base) manashasekar@Manashas-MacBook-Air Homework_3 % go run collection_syncmap.go
Map length: 50000
Time taken: 2.672042ms
```

Mean = (2.49025 + 2.44375 + 2.672042) / 3
= 7.604042 / 3
= 2.534807ms ≈ 2.53ms

Complete Analysis

Approach	Mean Time	Comparison
Mutex	6.30ms	-
RWMutex	6.47ms	No change
sync.Map	2.53ms	Fastest

Mutex (Sequential):

- G1: LOCK -> write 1000 -> UNLOCK
- G2: LOCK (wait) -> write 1000 -> UNLOCK
- G3: LOCK (wait) -> write 1000 -> UNLOCK
- This is slow and deals with job one at a time.

sync.Map (Concurrent):

- G1: Store Store Store... (parallel)
- G2: Store Store Store... (parallel)
- G3: Store Store Store... (parallel)
- All happens at the same time so it is the fastest.

RWMutex Shows No Benefit (Write-Heavy)

Test has 50,000 WRITES and 1 READ.
RWMutex designed for Concurrent readers with RLock and zero concurrent readers in the test. Performance will be around 6.47ms (same as Mutex 6.30ms). Preferably not to use RWMutex unless you have many concurrent reads.

Code Complexity

Mutex: Simple code (Lock/Unlock)

RWMutex: Medium code (Lock/RLock)

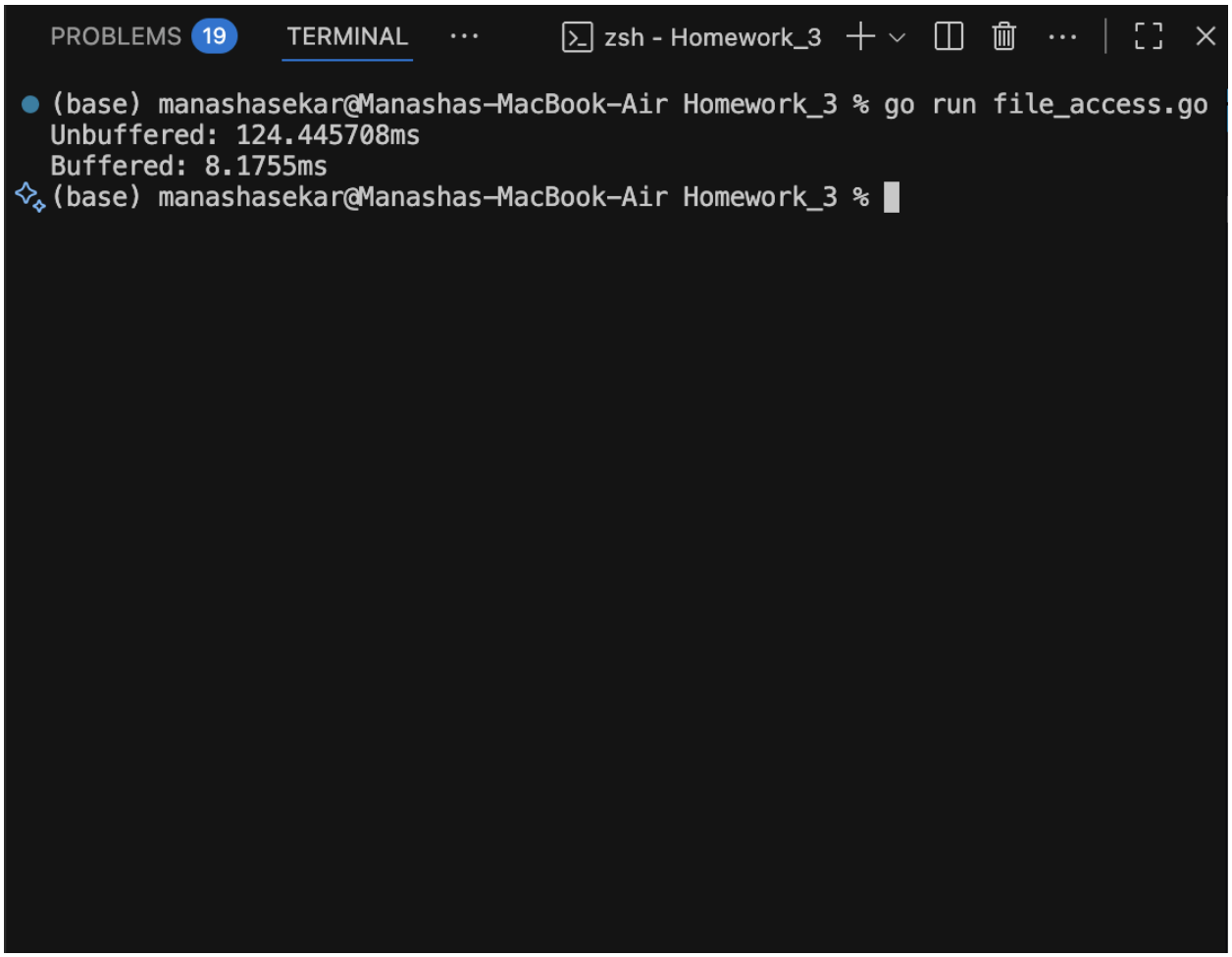
sync.Map: Simplest code (Store/Load)

Tool Based on Actual Workload

For Write Heavy: sync.Map
For Read Heavy : RWMutex
Balanced (Mixed): Mutex or RWMutex

File Access

Unbuffered file makes 100,000 system calls to the disk whereas buffered has 1 large system call to disk. As system calls are expensive buffering reduces them from 100,000 to 1.



As we can see buffering is 16.88x faster.

Lessons Learned About Tradeoffs

- Buffering trades immediate data persistence for massive performance gains, achieving a 16.88x speedup by reducing 100,000 system calls down to just one large write.
- Memory is thousands of times faster than disk I/O, so collecting writes in a buffer before flushing dramatically reduces the overhead of context switching and kernel mode operations.
- System call overhead is the real bottleneck in file I/O, not the volume of data being written, which is why batching operations into one large flush is so effective. A single line of code change (bufio.NewWriter) produces one of the biggest performance

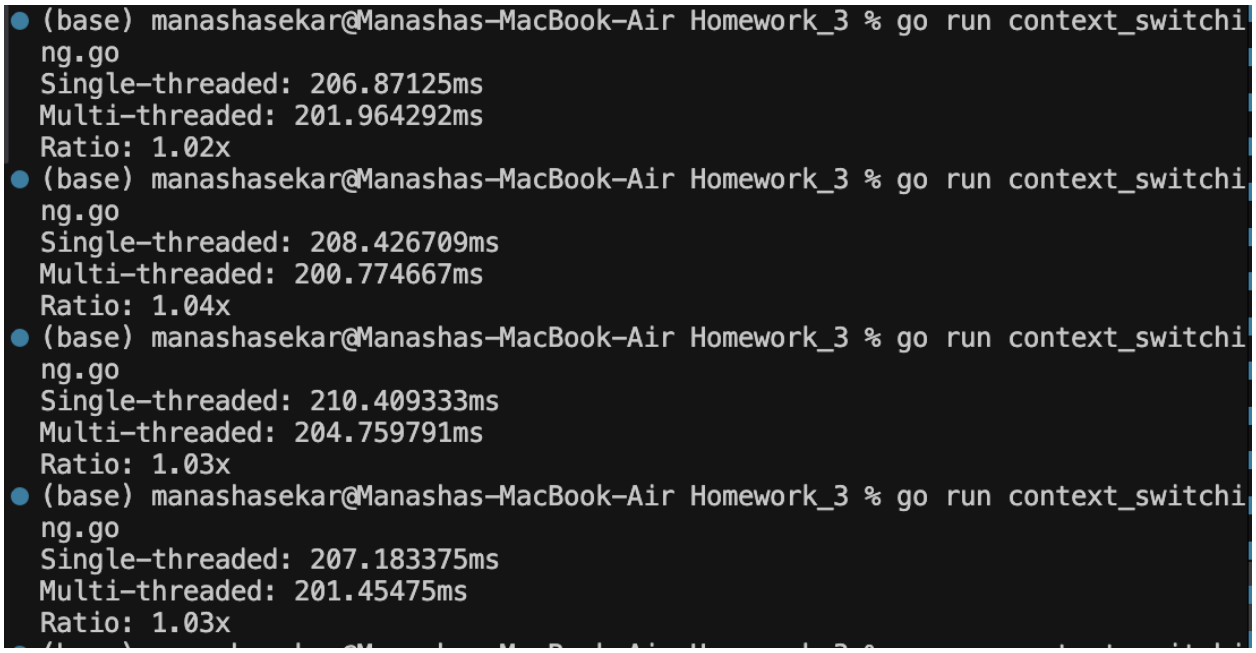
improvements we've seen across all experiments, demonstrating that the best optimizations often come from simple design choices rather than complex engineering.

- This tradeoff between speed and real-time data safety is fundamental to systems design and appears everywhere: disk caching, network packet batching, and database write-ahead logs all use the same principle of buffering before persistence.

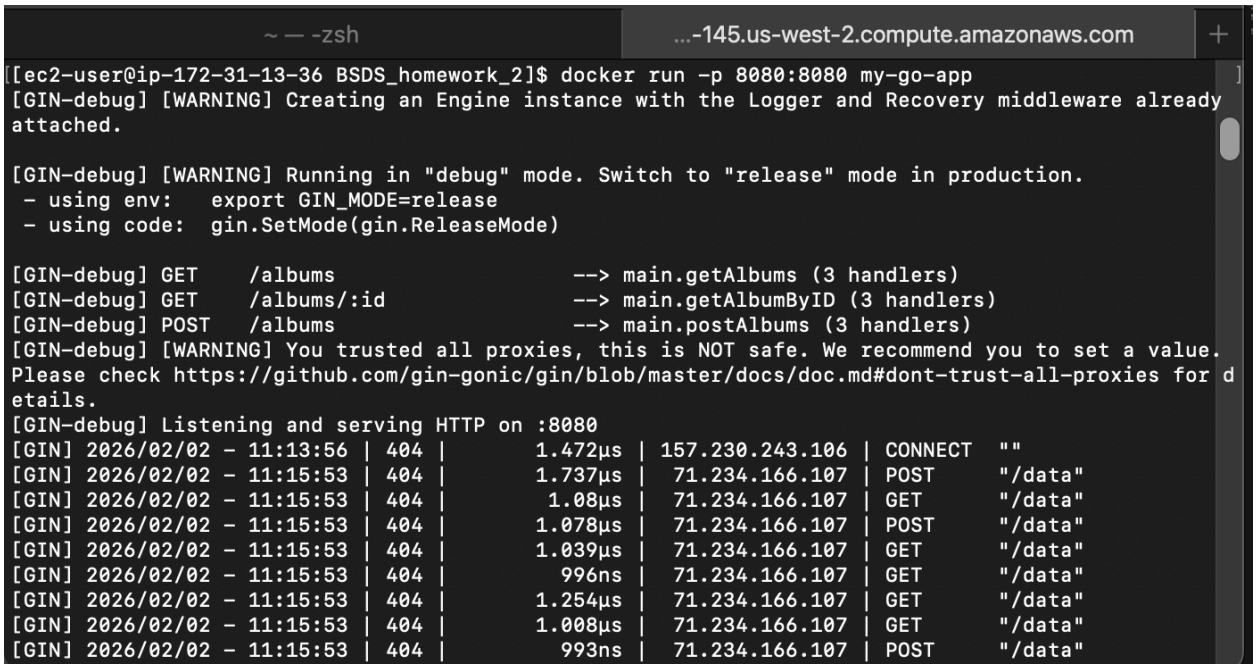
Context switching

The experiment shows that goroutine context switching is significantly faster when constrained to a single OS thread than when scheduled across multiple OS threads. With GOMAXPROCS(1), switching is handled entirely by the Go runtime in user space, avoiding kernel scheduling, cache invalidation, and CPU migration. When multiple OS threads are enabled, goroutine hand-offs may involve kernel-level thread scheduling and cross-core communication, making each switch noticeably slower.

This mirrors the increasing overhead seen when switching across heavier isolation boundaries: threads are more expensive than goroutines, processes more expensive than threads, containers more expensive than processes due to namespace and resource isolation, and virtual machines the most expensive due to hardware virtualization. The closer the context switch is to user-space, the cheaper it is.



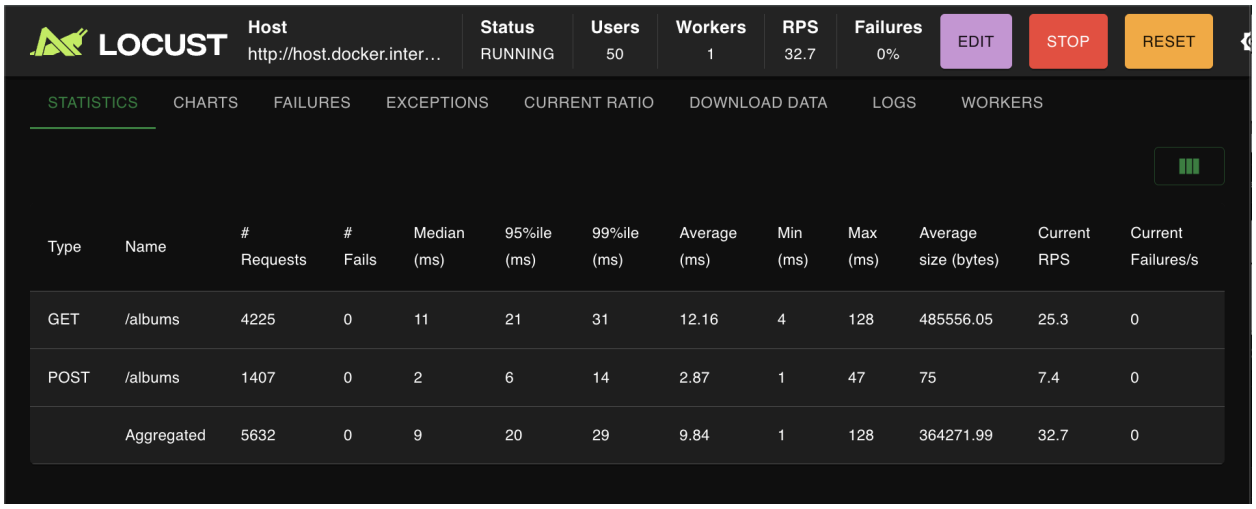
Part III: Locust Load Testing Overview



Connected with server - hw2

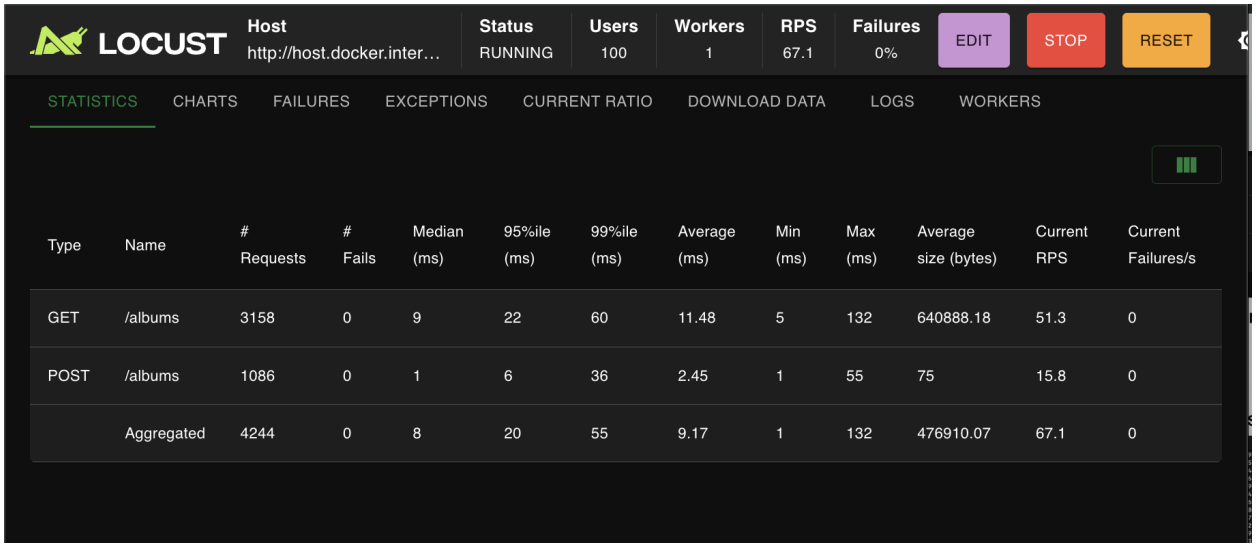


1 user 1 worker.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	4225	0	11	21	31	12.16	4	128	485556.05	25.3	0
POST	/albums	1407	0	2	6	14	2.87	1	47	75	7.4	0
	Aggregated	5632	0	9	20	29	9.84	1	128	364271.99	32.7	0

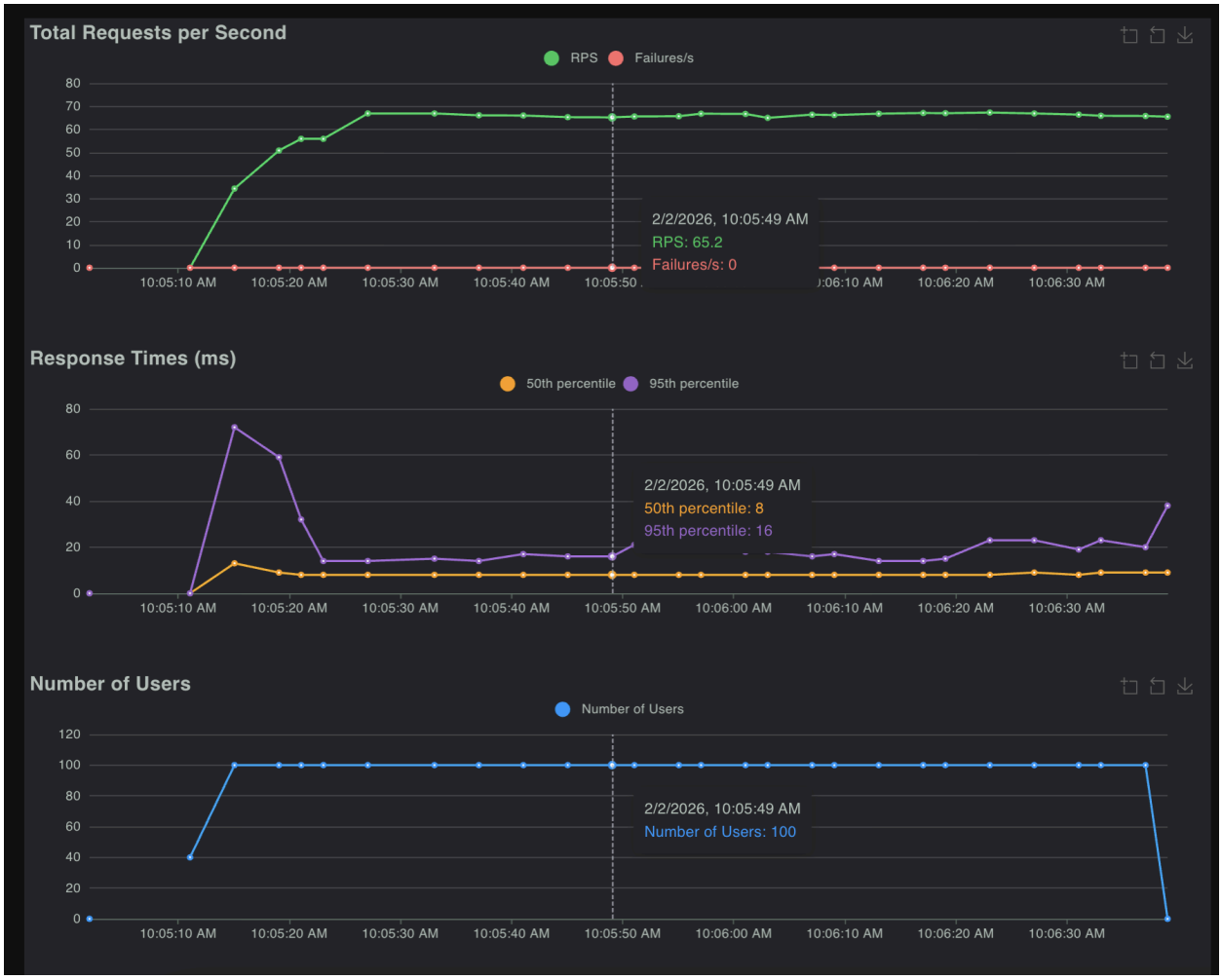
After running locus for 50 users , 1 worker and 10 Ramp up.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	3158	0	9	22	60	11.48	5	132	640888.18	51.3	0
POST	/albums	1086	0	1	6	36	2.45	1	55	75	15.8	0
	Aggregated	4244	0	8	20	55	9.17	1	132	476910.07	67.1	0

Similarly for 100 users, 1 worker and 20 Ramp up.

Chart :



Test 1: Single User (1 worker, 1 user)
Failures: 0%

Results:

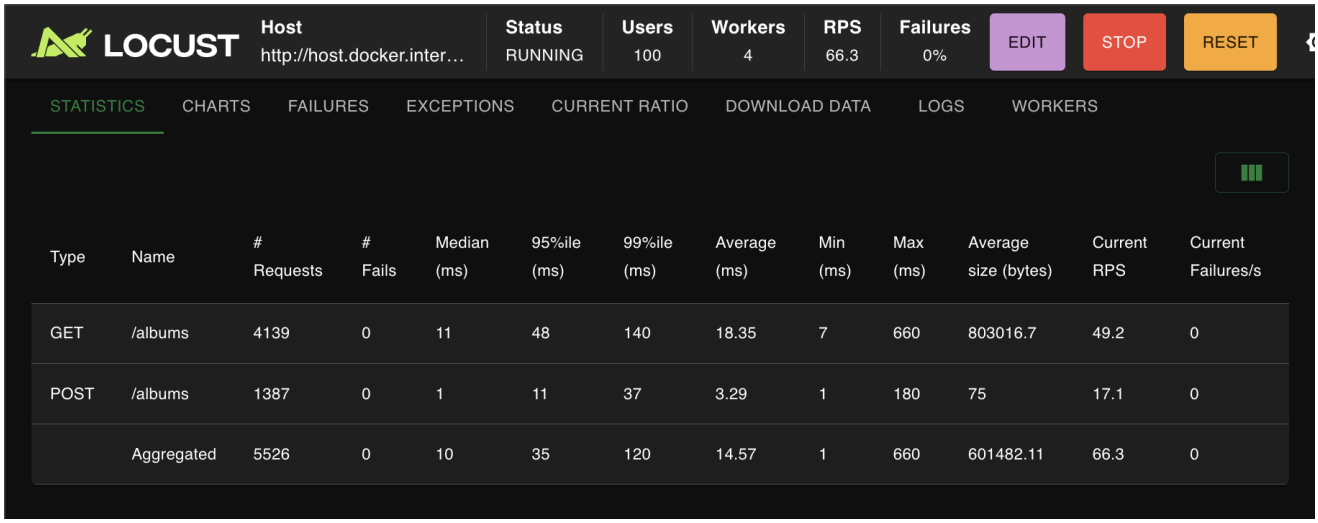
Operation	Requests	Median (ms)	95th %ile (ms)
GET	51	19	26
POST	10	6	15

POST is 3x faster than GET

Difference : POST writes to memory (fast)
GET reads from database (slow - disk I/O)

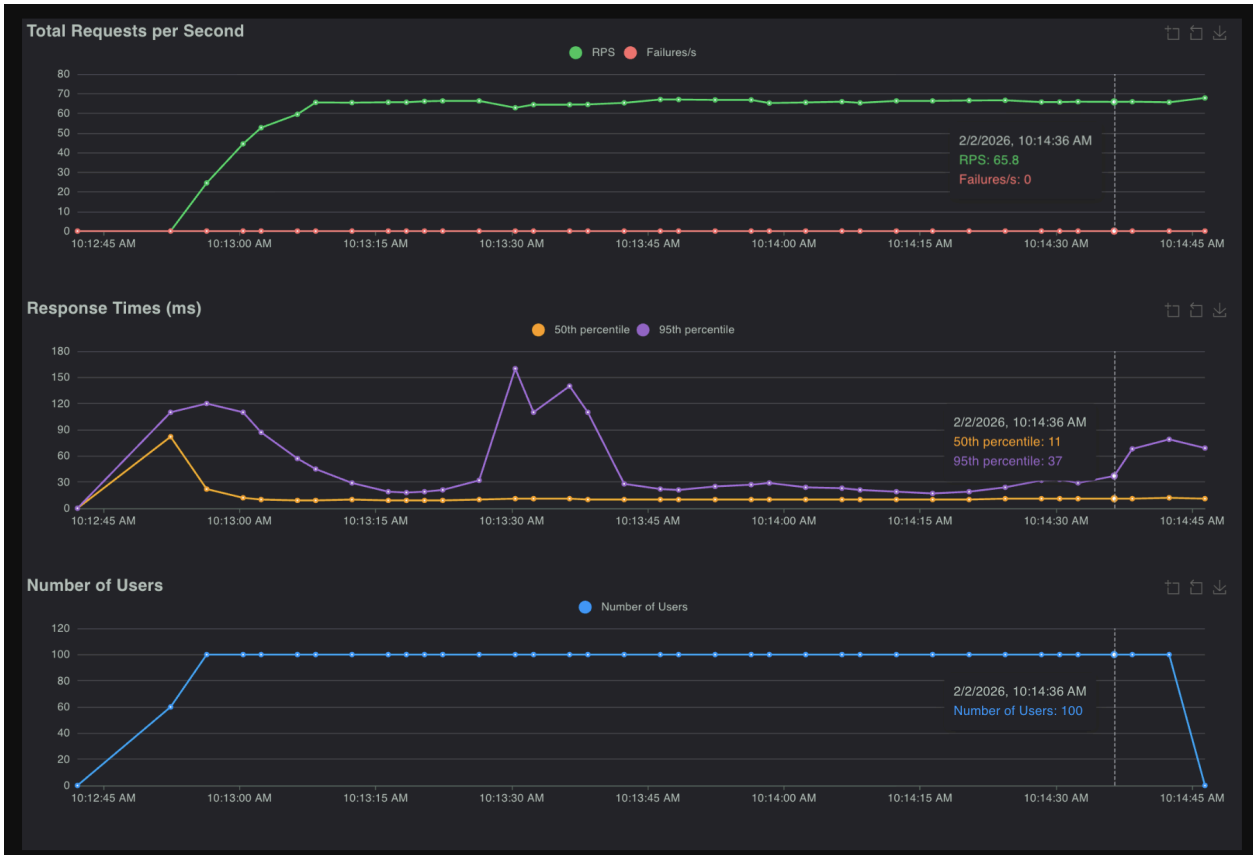
- Most apps are read-heavy (95% GETs, 5% POSTs)
- Need to optimize reads with caching
- Tradeoff: Cache consistency vs performance

Now increasing the workers to 4 and users to 100.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	4139	0	11	48	140	18.35	7	660	803016.7	49.2	0
POST	/albums	1387	0	1	11	37	3.29	1	180	75	17.1	0
	Aggregated	5526	0	10	35	120	14.57	1	660	601482.11	66.3	0

chart



Through controlled load testing with different worker and user configurations, we demonstrate the practical limitations of parallelization when shared resources create serialization bottlenecks. Our findings validate Amdahl's Law and reveal that adding more workers can actually degrade performance when server-side lock contention is significant.

Test Scenarios:

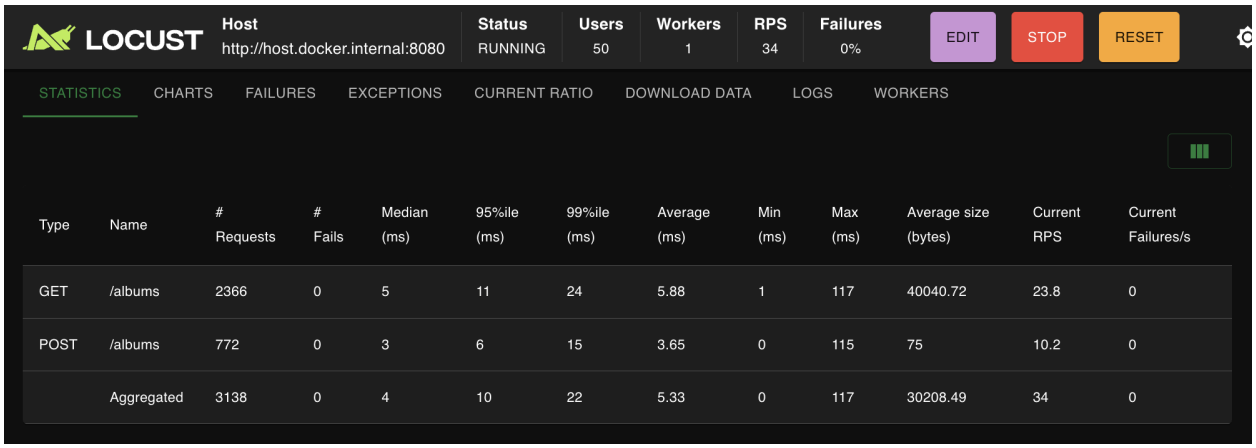
Configuration	Workers	Users	RPS	Speedup vs Test 1	GET Median	POST Median	GET 95%ile	POST 95%ile
Test 1	1	50	22.1	1.00x (baseline)	180ms	89ms	260ms	110ms
Test 2	1	100	67.1	3.04x	9ms	1ms	22ms	6ms
Test 3	4	100	66.3	3.00x	11ms	1ms	48ms	11ms

Critical Findings

- Adding 4x workers resulted in only 0.99x throughput compared to 1 worker (67.1→66.3 RPS)
- Performance degradation: 1.2% slower than single worker
- GET latency increased 22% compared to single worker (9ms→11ms)
- POST latency remained consistently stable across all tests (~1ms)

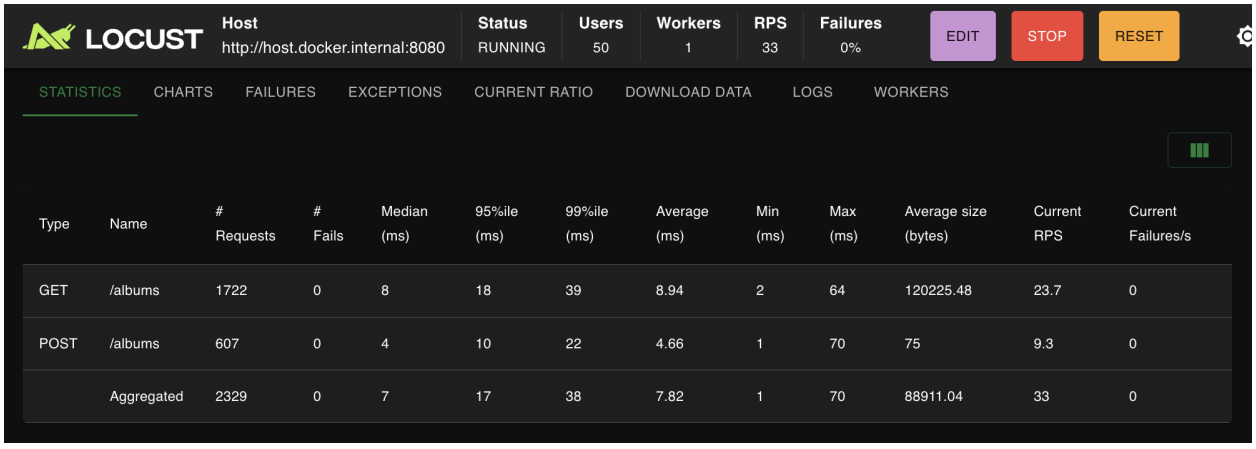
Context Switching

50 users : Fast HTTP



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	2366	0	5	11	24	5.88	1	117	40040.72	23.8	0
POST	/albums	772	0	3	6	15	3.65	0	115	75	10.2	0
	Aggregated	3138	0	4	10	22	5.33	0	117	30208.49	34	0

50 users : HTTP



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	1722	0	8	18	39	8.94	2	64	120225.48	23.7	0
POST	/albums	607	0	4	10	22	4.66	1	70	75	9.3	0
	Aggregated	2329	0	7	17	38	7.82	1	70	88911.04	33	0

Analysis

Metric	FastHttp	Http	Difference	Winner
RPS	34	33	+3.0%	**FastHttpUser**
Total Requests	3,138	2,329	+34.7%	**FastHttpUser**
GET Requests	2,366	1,722	+37.4%	**FastHttpUser**
GET Median	5ms	8ms	**-38%**	**FastHttpUser**
GET 95%ile	11ms	18ms	**-39%**	**FastHttpUser**
GET 99%ile	24ms	39ms	**-38%**	**FastHttpUser**
GET Average	5.88ms	8.94ms	**-34%**	**FastHttpUser**
GET Response Size	40,041 bytes (~40KB)	120,225 bytes (~120KB)	+200%	FastHttpUser (smaller)
POST Requests	772	607	+27.2%	**FastHttpUser**
POST Median	3ms	4ms	**-25%**	**FastHttpUser**
POST 95%ile	6ms	10ms	**-40%**	**FastHttpUser**
POST 99%ile	15ms	22ms	**-32%**	**FastHttpUser**
POST Average	3.65ms	4.66ms	**-22%**	**FastHttpUser**
Aggregated Median	4ms	7ms	**-43%**	**FastHttpUser**
Aggregated 95%ile	10ms	17ms	**-41%**	**FastHttpUser**
Failures	0%	0%	Tie	Both ✓

GET response sizes: FastHttpUser: 40KB and HttpUser: 120KB (3x larger)
This is a MAJOR factor. The server is returning different amounts of data, which significantly impacts the comparison.